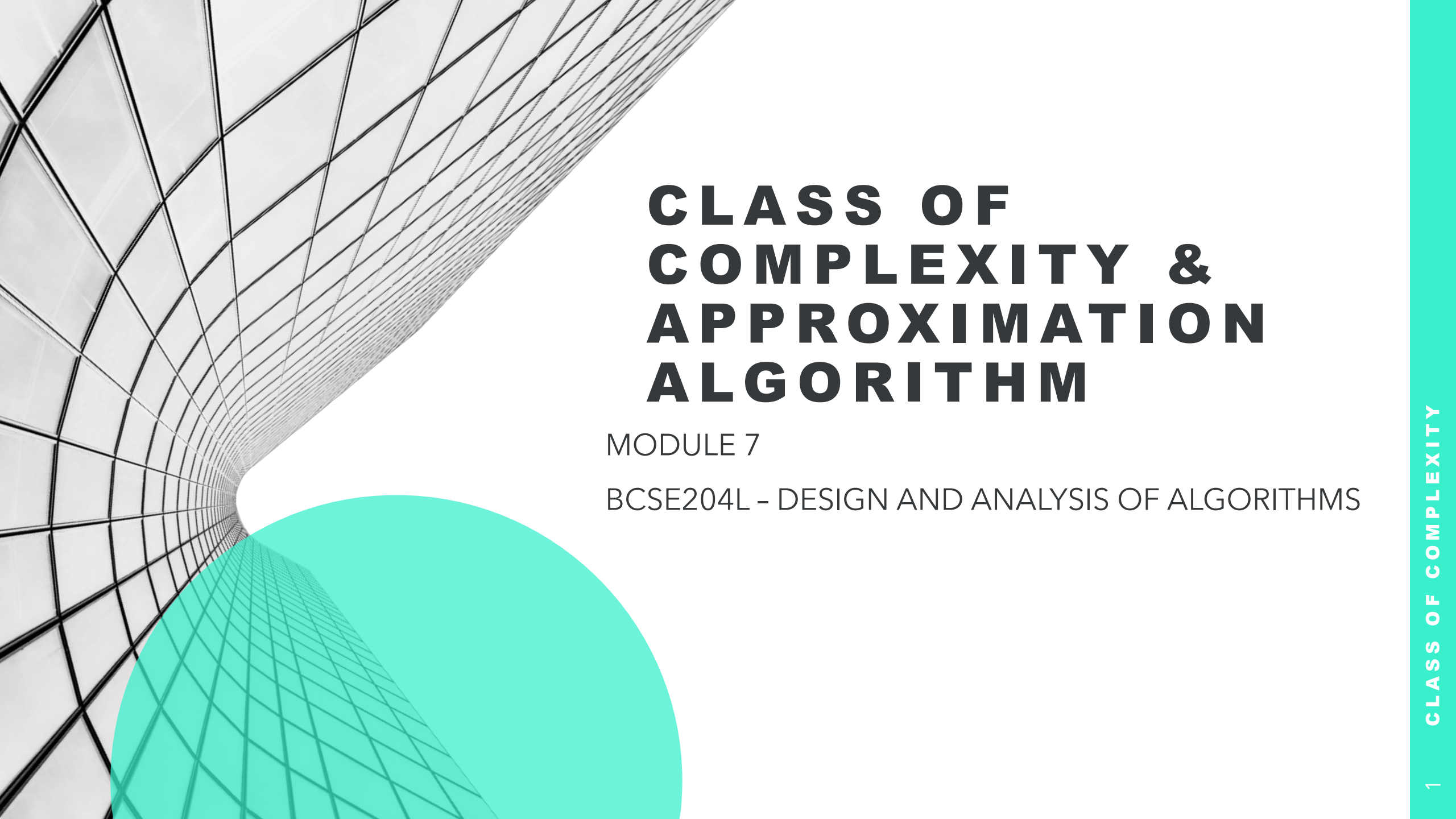




CLASS OF COMPLEXITY & APPROXIMATION ALGORITHM

MODULE 7

BCSE204L - DESIGN AND ANALYSIS OF ALGORITHMS

WHAT IS COMPUTATIONAL COMPLEXITY?

- The **computational Complexity Theory** (or Computability Theory or Theory of Computation), on the other hand, **deals with the solvability of a given problem.**
- It deals with the issues of deciding which problems can be computed and which cannot.

SOLVABILITY

- A **computable problem** implies that a problem has an algorithm with which the **problem can be solved**.
- A **non-computable problem** implies that the **problem has no algorithm**
- Both focus computability and computational complexity theories is to **analyze or evaluate the hardness of a given problem**.
- **Hard** or **Easy** refer to whether the problem are difficult to compute or not.
- **Hardness** indicates the cost of solving a problem or its solvability within given limited resources such as time or space.

UPPER BOUNDS & LOWER BOUND

➤ Upper Bound

- It is the measure of the **maximum computational effort required to solve the given problem.**
- The **worst case time complexity** of an algorithm **refers to the maximum amount of time taken** by the algorithm to produce the correct output for any valid input.
- The worst case complexity of an algorithm is also **indicated as a function of the input size n**
- Eg. If there are 3 algorithms are available for a given problem, then the worst case complexity is the maximum of the resource (time and space) cost for these 3 algorithms

UPPER BOUNDS & LOWER BOUND

➤ Lower Bound

- It is the measure of the **minimum amount of work needed to solve the problem.**
- It indicates that the given algorithm will use no less than the lower bound-specified resource.
- Resource – memory, CPU time, bandwidth etc.
- The lower bound theory **is applied to the given problem and not to the algorithms of given problem.**
- Set of all algorithms S to solve problem A , then lower bound is given as

$$\min_{A \in S} \{ \text{Worst cost of all algorithm of Set } S \}$$

ALGORITHMIC GAP

- The **difference between Upper and lower bound** is called **Algorithmic gap**.
- If gap is **large**, then **faster algorithm is possible or still better lower bound needs to be obtained**.
- Finding lower bound is real challenge in the computational complexity theory.

METHODS OF FINDING LOWER BOUNDS

➤ Trivial Lower Bound:

- It is obtained by **counting the number of items that are processed by an algorithm as inputs and the number of items that are produced after processing by the algorithm**
- Eg. Trivial lower bound for matrix addition is number of items that needs to be processed is $2n^2$. Therefore, the lower bound is given as $\Omega(n^2)$.

METHODS OF FINDING LOWER BOUNDS

- **Information theoretic Arguments**
- Used to **find the lower bound based on amount of information produced by the algorithm**
- It is very effective for searching and sorting algorithms
- It use decision tree to find lower bounds
 - The node of decision tree have only two outcomes - yes or no.
 - The decision tree processes a query and produce two outcomes.
 - It can be used as computing models to analyze algorithms and can serve as a tool to find the lower bound.

METHODS OF FINDING LOWER BOUNDS

➤ Information theoretic Arguments

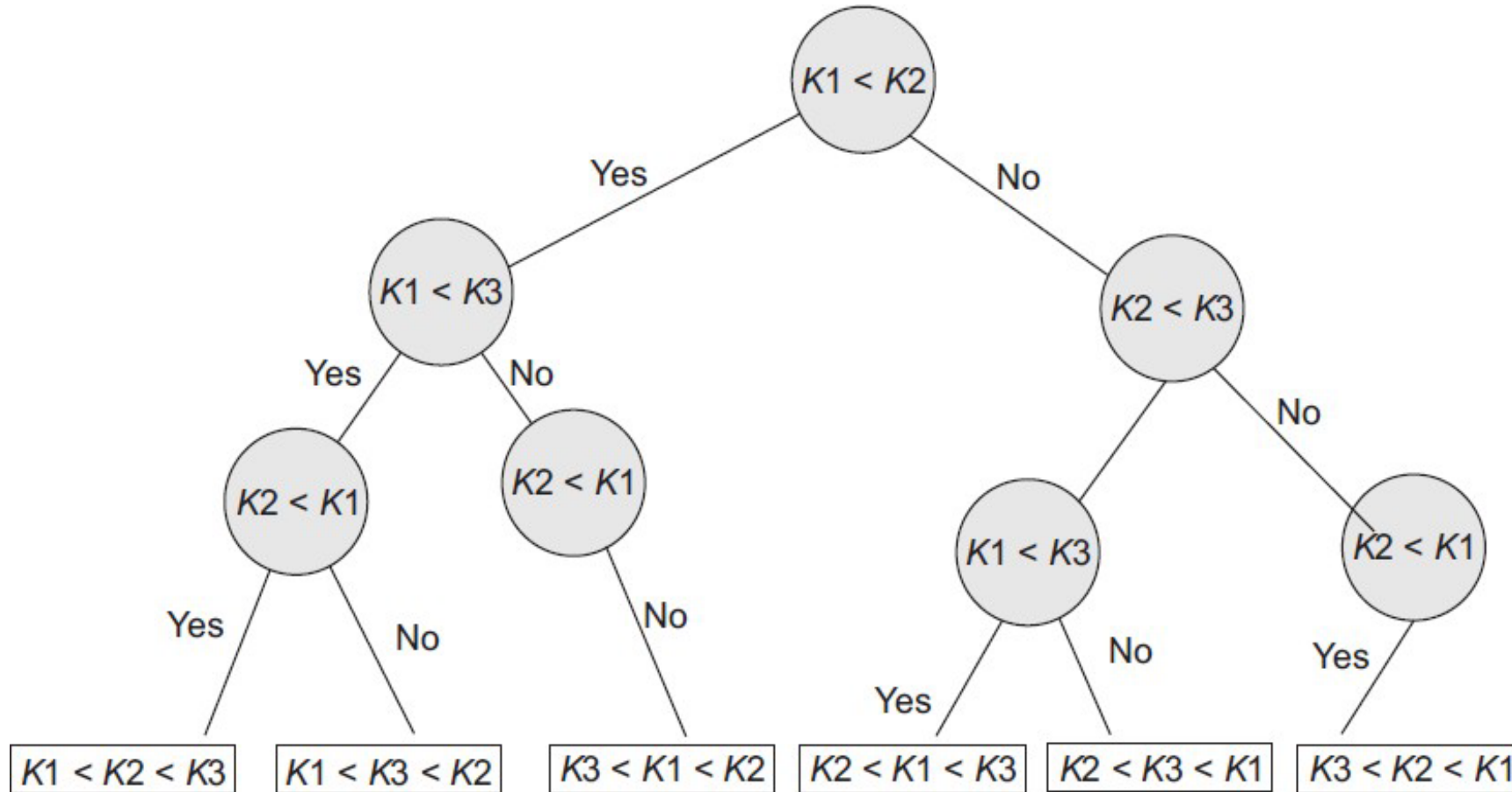


Fig. 18.1 Decision tree

METHODS OF FINDING LOWER BOUNDS

➤ Information theoretic Arguments

- Let us consider 3 numbers - K1, K2, K3 for sorting and to find the lower bound for comparison based sorting problem using the decision tree.
- The decision tree produces 6 possible permutation for 3 elements.
- So for n elements, we can have n! possible outcome.
- The minimum number of comparison is **$T(n) \geq \log_2 n!$**
- Using sterling approximation **$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$**
- Therefore **$T(n) \geq \log \left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \right)$
 $\geq \log \sqrt{2\pi n} + n \log n - n \log e \approx \Omega(n \log n).$**

METHODS OF FINDING LOWER BOUNDS

➤ Reduction Argument

- It is a **proof technique** which show that the solution of a problem can be used to solve a different problem.
- Let us assume that there are two problem A and B.
 - Assume algorithms are available for solving B alone.
 - An algorithm is designed to solve A using algorithm of B as subroutine.
 - To find lower bound by reducing another problem whose lower bound is known, which helps to determine **the hardness of the given problem.**
 - To prove **B** harder than **A**, reduce **B** to **A** as $A \leq_p B$

METHODS OF FINDING LOWER BOUNDS

➤ Reduction Argument

- Problem A and B has inputs $x_1, x_2, \dots, x_n$ and output as $y_1, y_2, \dots, y_n$. Assume A can be used to solve another problem B . Then reduction procedure can be
 - Convert instance of problem A into instance of problem B using a polynomial algorithm that takes only a linear time.
 - Call the algorithm available for problem B
 - Convert the output of problem B to that of Problem A .
- The reduction $A \leq_p B$ implies the following conditions:
 - If problem B is easy, A is also easy to solve.
 - If problem B is hard, then A is not harder than B . And the upper bound of A is at most the upper bound of B and lower bound of B is at least the lower bound of A

CATEGORIZATION OF ALGORITHMS

- Polynomial Algorithm
- Exponential Algorithm

CATEGORIZATION OF ALGORITHMS

➤ Polynomial Algorithm

- Polynomial $p(n)$ of degree n .
- An algorithm is called a polynomial algorithm **if the upper bound is $O(p(n))$** . Eg. Searching, sorting etc.
- Polynomial algorithm having complexity n^{100} are complex as exponential algorithms, in reality, algorithm having a large polynomial degree.

CATEGORIZATION OF ALGORITHMS

➤ Polynomial Algorithm

- It is **easy to solvable** because of following reasons
 - All machine are inherently polynomial - all constant of the polynomial vary from machine to machine.
 - Operations of polynomial are closed under addition and composition.
 - Polynomial algorithms takes finite time and use finite resources.
- Polynomial algorithms are desirable algorithms for any given task.

CATEGORIZATION OF ALGORITHMS

➤ Exponential Algorithm

- If **algorithm's complexity is not polynomial**.
- Algorithms whose complexity is $O(n!)$ or $O(2^n)$ are all exponential algorithms.
- These are **infeasible algorithms**, that **cannot be solved within a reasonable amount of time**.
- It is characterized by a phenomenon called combinatorial explosion, which means that the **algorithm output grows alarmingly when the input size increased**.
- Eg. Tower of Hanoi and TSP.

COMPUTATIONAL COMPLEXITY CLASS

- Problem are classified into two types based on the hardness
- **Analytical hard problems**
 - Do not have a proper mathematical formulation
 - Need enormous amount of information
 - Incomplete specifications,, partial problem statements etc. E.g. AI
- **Computational hard problems**
 - Intractable problems that cannot be solved using available algorithms that use a available resources.
 - Are categorized into two groups called **complexity classes**.
- **Complexity classes** - set of problem s that have similar runtime or rates growth
- Classified as **Class P** and **Class NP**

CLASS P PROBLEMS

- Problem for which polynomial time algorithms exist are said to belong to the complexity class **class P**.
- **P** stands for polynomial time.
- All the instance of the Class P in $O(p(n))$ time.
- Class P are considered easy problems, can be computed efficiently.
- E.g. searching, sorting and selection problems.

CLASS NP PROBLEMS

- Problem are said to be intractable, and do not have any efficient solution are called **class NP problems**.
- **NP** stands for non-deterministic polynomial, refers to problem that are solvable in polynomial time.
- It consists of decision problem for which there exists a verification algorithm that can verify the input decision problem in polynomial time.
- NP problem are of two types
 - **NP-hard** - are hard and similar to any other problem under class NP.
 - **NP-complete** - are hardest of the class NP problems that are in between class NP-hard and class P problems. The solution obtained for a single problem can be used to solve other problems belong to this.

NP-HARD PROBLEMS

- It is **not possible to solve all problems**
- For some problems, there is **no algorithm at all**.
- E.g. Halting problem by Alan Turing in 1936, halting undecidable problem that cannot be solved.
 - Halting problem stated as follows: given an algorithm **S**, is it possible to check whether problem **P** halts or not?
 - **stop()** - takes problem **P** as argument and halt if **stop()** return true else return false.

```
function stop( )  
if stop(P) then  
    print 'P halts'  
else  
    print 'P does not halt'
```

NP-HARD PROBLEMS

- Using this function once can write Program **S** as

Program **S**

```
Label: if (stop(P)) then  
    go to label1  
    else  
        Stop
```

- It can be observed that the program takes a Problem **P**, and if **stop(P)** is true, then the program loops forever; else the program stops.
- A crucial problem arises in a halting problem when the input for program **S** is itself. If so the program **S** is given as follows:
- If **stop(S)** is true then as per program logic, the loop forever or if **stop(S)** is false then it stops.
- This is contradiction, is called "**diagonal arguments**"
- Based on this, **halting problem is not solvable.**

Program **S**

```
Label: if (stop(S)) then  
    go to label1  
    else  
        Stop
```

NP-COMPLETE PROBLEMS

- It is **similar to NP-hard** problems, but **verification of the algorithms can be done in polynomial time**.
- Complete - given problem is hard as all other problems in the problem set, **if one problem is solvable then all other problem in the set can be solved**.
- It can **solve all problems of class NP**
- E.g. factoring problem and graph isomorphism

NP-COMPLETE PROBLEMS

- NP-complete— **if it is in NP and is as “hard” as any problem in NP.**
- NP-complete problem can be solved in polynomial time, then every problem in NP has a polynomial time algorithm.
- NP-complete problems **are intractable**, since given the wide range of NP-complete problems —**without anyone having discovered a polynomial time solution to any of them**—it would be truly a sounding if all of them could be solved in polynomial time.

NP-COMPLETE PROBLEMS

- **Overview of showing problems to be NP-complete**
- To demonstrate that a problem is NP-complete,
- we are **making a statement about how hard it is** (or at least how hard we think it is), **rather than about how easy it is.**
- We are not trying to prove the existence of an efficient algorithm, but *instead that no efficient algorithm is likely to exist.*
- In this way, NP-completeness proofs bear **$O(n \log n)$** time lower bound for any comparison sort algorithm and it differ from the decision-tree method.

NP-COMPLETE PROBLEMS

- 3 concepts for showing problem to be NP-complete
- **Decision problems vs. optimization problems**
- Many problems of interest are optimization problems, in which each feasible (i.e., "legal") solution has an associated value, and we wish to ***find a feasible solution with the best value.***
- Although NP-complete problems are confined to the realm of **decision problems**, we can take advantage of a convenient relationship between optimization problems and decision problems.
- We usually can **cast a given optimization problem as a related decision problem by imposing a bound on the value to be optimized.**

NP-COMPLETE PROBLEMS

- **3 concepts for showing problem to be NP-complete**
- **Decision problems vs. optimization problems**
 - **SHORTEST-PATH** is the single-pair shortest-path problem in an unweighted, undirected graph.
 - NP completeness applies directly not to optimization problems, however, but to decision problems, in which the answer is simply "yes" or "no"
- The relationship between an optimization problem and its related decision problem works in our favor when we **try to show that the optimization problem is "hard."**
- That is because the decision problem is in a sense "easier," or at least "no harder."

NP-COMPLETE PROBLEMS

➤ 3 concepts for showing problem to be NP-complete

➤ Reductions

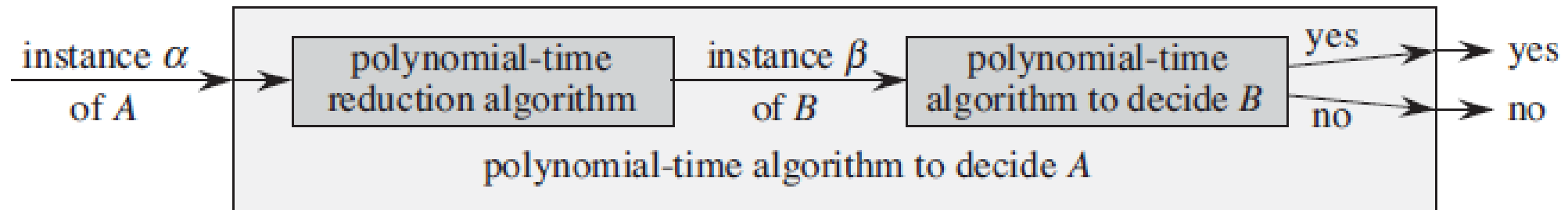
- Let us consider a decision problem A , which we would like to solve in polynomial time.
- To solve a different decision problem B in polynomial time. Finally, suppose that we have a procedure that transforms any instance α of A into some instance β of B with the following characteristics:
 - The transformation takes polynomial time.
 - The answers are the same. That is, the answer for α is "yes" if and only if the answer for β is also "yes."

NP-COMPLETE PROBLEMS

➤ 3 concepts for showing problem to be NP-complete

➤ Reductions

- A procedure a polynomial-time reduction algorithm used to solve problem A
 - Given an instance α of A , used a polynomial-time reduction algorithm to transform it to an instance β of B .
 - Run polynomial-time decision algorithm for B on the instance β .
 - Use the answer for β as the answer for α



NP-COMPLETE PROBLEMS

- 3 concepts for showing problem to be NP-complete
- Reductions
- “Reducing” solving problem **A** to solving problem **B**, we use the “easiness” of **B** to prove the “easiness” of **A**.
- NP-completeness is about **showing how hard a problem is rather than how easy it is**, we use polynomial-time reductions in the opposite way to show that a problem is NP-complete.
- NP-completeness, we cannot assume that there is absolutely no polynomial time algorithm for problem **A**. The proof methodology is similar, however, in that we prove that problem **B** is NP-complete on the assumption that problem **A** is also NP-complete.

SATISFIABILITY PROBLEM

- Stephen Arthur Cook and L.A. Levin in 1973 proved that the **satisfiability problem (SAT)** is NP-complete.
- **SAT is a significant problem and can be stated as follows**
 - Given Boolean expression F having n variables $x_1, x_2, \dots, x_n$ and Boolean operators, is it possible to have assignment for variables true or false such that the binary expression F is true?
 - This problem is known as formula-SAT.
 - A SAT takes a Boolean expression F and checks whether the given expression is satisfiable.
 - A Boolean expression is said to be satisfiable for some valid variables.

SATISFIABILITY PROBLEM

➤ Terminologies

- **Boolean Variable** – say x , that can have **only two values** (true or false)
- **Literal** – logical variable x is called positive and **negation of x** is called negative.
- **Clause** – Sequence of variable $x_1, x_2, \dots, x_n$ can be **separated by logical OR operator**.

SATISFIABILITY PROBLEM

➤ Terminologies

- **Expression** - combine all the preceding clauses using a Boolean operator to form an expression.
- **CNF form** - An expression is in Conjunctive Normal Form if set of clause are separated by AND operator, while literal are connected by OR operator. $f = (x_1 \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee \overline{x_3} \vee x_2)$
- **3-CNF** - an expression is said to be in 3-CNF if it is in the conjunctive normal form, and every clause has exact three literals.

SATISFIABILITY PROBLEM

- A general definition of the SAT problem.
- A given Boolean expression, determining **if there exists a truth assignment to its variables for which the expression is true is defined as the satisfiability problem.**
- Let's take an example to discuss further.

a	b	c	$(a \wedge b) \vee c$	$(a \wedge \neg a) \vee (c \wedge \neg c)$
1	1	1	1	0
0	1	1	1	0
0	0	1	1	0
0	1	0	0	0
1	0	1	1	0
1	0	0	0	0
1	0	1	1	0
0	0	0	0	0

SATISFIABILITY PROBLEM

- To test whether there exists a truth assignment to variables a, b, c , we've tested all possible assignments.
- There are 5 satisfying truth assignments for the expression $(a \wedge b) \vee c$ and hence it is satisfiable.
- But the expression $(a \wedge \neg a) \vee (c \wedge \neg c)$ is false for all possible truth assignments and hence it is unsatisfiable.
- This takes $O(2^3)$. This is called the **truth table method**. If there n variables then method takes $O(2^n)$ time to determine expression result

a	b	c	$(a \wedge b) \vee c$	$(a \wedge \neg a) \vee (c \wedge \neg c)$
1	1	1	1	0
0	1	1	1	0
0	0	1	1	0
0	1	0	0	0
1	0	1	1	0
1	0	0	0	0
1	0	1	1	0
0	0	0	0	0

SAT VARIANTS

- **2-SAT** - every clause has two literals and is solvable in a polynomial-time problem.
- Pick a variable a , and assign true. We assign true to the other variable of all the clauses with $\neg a$. If all clauses are satisfied, then we pick the remaining clauses which don't have a or $\neg a$ and continue the process.
- On the other side, if the all clauses $\neg a$ are not satisfied, then we assign false to a and repeat the above process. If all the clauses containing a and $\neg a$ are all not satisfied, then the **2-SAT is unsatisfiable.**
- **Application of 2-SAT**
 - To identify an undirected graph that can be represented as a complete bipartite graph and independent set.
 - To identify the business relationships among autonomous subsystems of the internet.

SAT VARIANTS

➤ Weighted 2-SAT

- Given a **CNF with 2 literals per clause and an integer k** , the problem of determining if there is a truth assignment with at most k variables set to true for which the CNF is satisfiable.
- This problem is NP-Complete.
- Many algorithms use the weighted 2-SAT concept for solving the vertex cover problem.

SAT VARIANTS

➤ 3-SAT

- In the 3-SAT problem, **each clause has at most 3 literals**.
- This problem is NP-Complete.
- The 3-SAT problem is part of the Karp's 21 NP-complete problems and is used as the starting point to prove that the other problems are also NP-Complete.
- One example is the independent set problem. The **Independent Set Problem** can be shown to be NP-Complete by showing that the **3-SAT is polynomial reducible to an independent set problem**.

SAT VARIANTS

➤ Exactly one 3-SAT

- **Each clause in the 3-SAT problem has at least 1 literal** that must be true.
- But in Exactly one 3-SAT problem, we determine if the CNF is satisfiable where exactly one of the three literals in every clause is true and the other two are false.
- This problem is NP-Complete.
- Exactly one 3-SAT is a known NP-Complete problem, and it's used in reduction to prove other problems NP-complete.

COOK-LEVIN THEOREM

- Cook-Levin theorem which shows how to prove that the SAT is an NP-Complete problem.
- **The statement of Cook-Levin theorem is the Boolean satisfiability problem is NP-complete.**
- For a given truth assignment, we can verify if the CNF or Boolean expression is satisfiable or not in polynomial time.
- If a decision problem can be verified in polynomial time by a **deterministic Turing machine**, then it can be solved in polynomial time by a **non-deterministic algorithm**.
- SAT is in NP if there is a non-deterministic Turing machine that can solve it in polynomial time.

COOK-LEVIN THEOREM

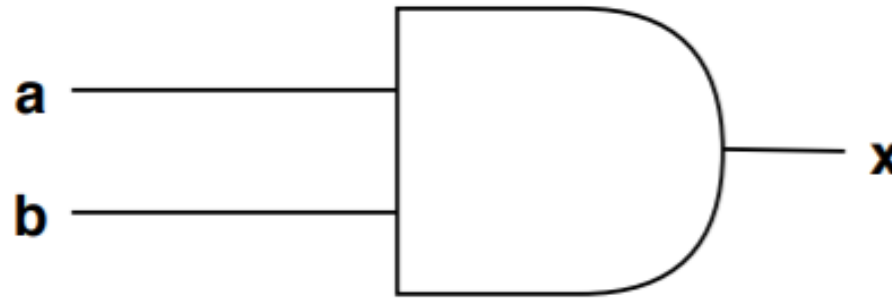
- **If any problem in NP can be reduced to an SAT problem in Polynomial-time, then it's NP-Complete.**
- To prove by taking any language $L \in NP$ and reducing it to SAT in polynomial time.
- Because $L \in NP$, there exists a verifier V which can verify the decision problem L in polynomial time.
- Let w be an input on which verifier V checks if the output is a *yes* or a *no* or (1 or 0).
- Because V works in polynomial time, at every time step, the algorithm makes the hardware affect a constant number of memory units.

COOK-LEVIN THEOREM

- After t time steps, the algorithm affects t memory units.
- Represent each of these operations at every time step using AND, OR, NOT logical gates. Hence it is possible to build a circuit in polynomial for any language $L \in NP$.
- This would be a polynomial time in the number of bits present in input w . The decision problem L now reduces to determining if there is an input w for which the circuit built above outputs 1 (true).
- The CNF equivalents of AND, OR, and NOT gates are shown below. This reduction is done using De Morgan's laws, **material implication**, and **double negation**.

COOK-LEVIN THEOREM

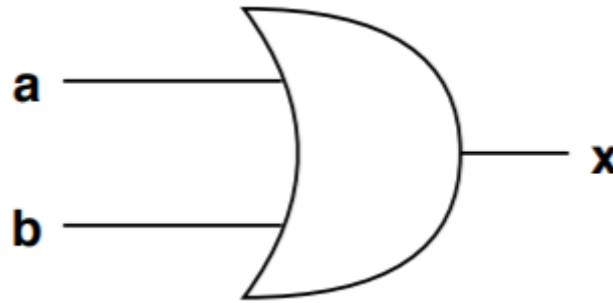
- Let's start with AND gate. Let a , b be the inputs, x be the output:



- The functionality of the above AND gate in CNF form is:
 - $x \Leftrightarrow (a \wedge b)$
 - $(x \Leftrightarrow (a \wedge b)) \wedge ((a \wedge b) \Rightarrow x)$
 - $(\neg x \vee (a \wedge b)) \wedge (\neg(a \wedge b) \vee x)$
 - $(\neg x \vee a) \wedge (\neg x \vee b) \wedge (\neg a \vee \neg b \vee x)$

COOK-LEVIN THEOREM

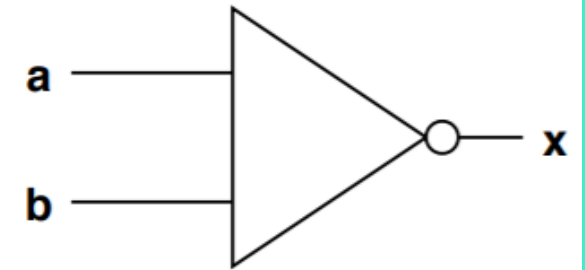
- Let's start with OR gate. Let a, b be the inputs, x be the output:



- The functionality of the above OR gate in CNF form is:
 - $x \Leftrightarrow (a \vee b)$
 - $(x \Leftrightarrow (a \vee b)) \wedge ((a \vee b) \Rightarrow x)$
 - $(\neg x \vee (a \vee b)) \wedge (\neg(a \vee b) \vee x)$
 - $(\neg x \vee a \vee b) \wedge (\neg a \vee \neg b \vee x)$

COOK-LEVIN THEOREM

➤ Let's start with NOT gate. Let a be the inputs, x be the output:



➤ The functionality of the above NOT gate in CNF form is:

➤ $a \Leftrightarrow \neg x$

➤ $(\neg x \Rightarrow a) \wedge (a \Rightarrow \neg x)$

➤ $(a \vee x) \wedge (\neg a \vee \neg x)$

➤ From the above gates, we can observe that we can convert the circuit into an equivalent CNF form.

➤ Hence all NP-Hard problems can be reduced to CNF, which means, they can be reduced to an SAT problem. Hence the **SAT is NP-Complete.**

3 - S A T

- 3-SAT defines the problem of determining whether a given CNF, with each clause containing at most 3 literals, is satisfiable or not.
- The 3-SAT problem is simpler than 2-SAT as it seeks to solve the 2-SAT problem **where there can be at most three variables in each parenthesis in the Boolean expression.**
- Still, there is no polynomial-time algorithm exists for 3-SAT.

3-SAT

- Let's see an example. We'll take an SAT problem and make it a 3-SAT.

$$\emptyset = (x_1 \vee x_2 \vee \neg x_4 \vee \neg x_5) \wedge (\neg x_1 \vee x_4 \vee x_5 \vee \neg x_6) \wedge (x_6 \vee x_7) \wedge (\neg x_2 \vee x_3)$$

- $\emptyset$ is satisfiable for truth values: $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1, x_5 = 0, x_6 = 0, x_7 = 1$

- We can introduce two new variables y_1, y_2 to make the given SAT problem to a 3-SAT problem:

$$\emptyset = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee \neg x_4 \vee \neg x_5) \wedge (\neg x_1 \vee x_4 \vee y_2) \wedge (\neg y_2 \vee x_5 \vee \neg x_6) \\ \wedge (x_6 \vee x_7) \wedge (\neg x_2 \vee \neg x_3)$$

- $\emptyset$ is satisfiable for truth values $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1, x_5 = 0, x_6 = 0, x_7 = 1, y_1 = 0, y_2 = 0$. In fact, for either values of y_1 and y_2 , the **3-CNF is satisfiable**.

NP-COMPLETENESS OF 3-SAT

- In this section, let's see how we can **prove that 3-SAT is NP-Complete**.
- We can convert any problem $L \in NP$ into an SAT problem in polynomial time. That is, we can express it as a Boolean formula and can convert every Boolean formula into its corresponding CNF form.
- SAT to 3-SAT reduction takes polynomial time. That is the corresponding CNF to 3-CNF takes polynomial time. Let the CNF representation of the original SAT problem is:
- $\emptyset = C_1 \wedge C_2 \wedge \dots \wedge C_n$ where $\emptyset$ is satisfiable if every clause C_i is satisfiable.

NP-COMPLETENESS OF 3-SAT

- Without loss of generality, let assume that the clause C_r has more than 3 literals:
- $C_r = (a_1 \vee a_2 \vee \dots \vee a_m), m > 3$ where each a_i is picked from a set of literals: $x_1, x_2, \dots, x_m, \neg x_1, \neg x_2, \dots, \neg x_m$
- Let $A = a_3 \vee a_4 \vee \dots \vee a_m, C_r = a_1 \vee a_2 \vee A$
- Define C'_r as following $C'_r = (a_1 \vee a_2 \vee y) \wedge (\neg y \vee A)$
- where y is a new variable and we introduce it to make a clause with 3 literals. This same procedure can be applied repeatedly to the second clause in C'_r until there are no more clauses with more than three literals remaining.

NP-COMPLETENESS OF 3-SAT

- Now we need to prove that C'_r and C_r are equisatisfiable.
- For case 1 we need to show C'_r is satisfiable, then C_r is satisfiable. Let's start this proof:
- C'_r is satisfiable, then C_r is satisfiable $\Rightarrow (a_1 \vee a_2 \vee y) = 1, (\neg y \vee A) = 1$
if $y = 0$: $C'_r = (a_1 \vee a_2 \vee 0) \wedge (1 \vee A) = (a_1 \vee a_2) \wedge (1) = (a_1 \vee a_2) = 1$
if $y = 1$: $C'_r = (a_1 \vee a_2 \vee 1) \wedge (0 \vee A) = (A) = 1$
- For either values of y , for C'_r to be satisfiable, either $(a_1 \vee a_2) = 1$ or $(A) = 1$, we can say that $C_r = 1$ if $C'_r = 1$.

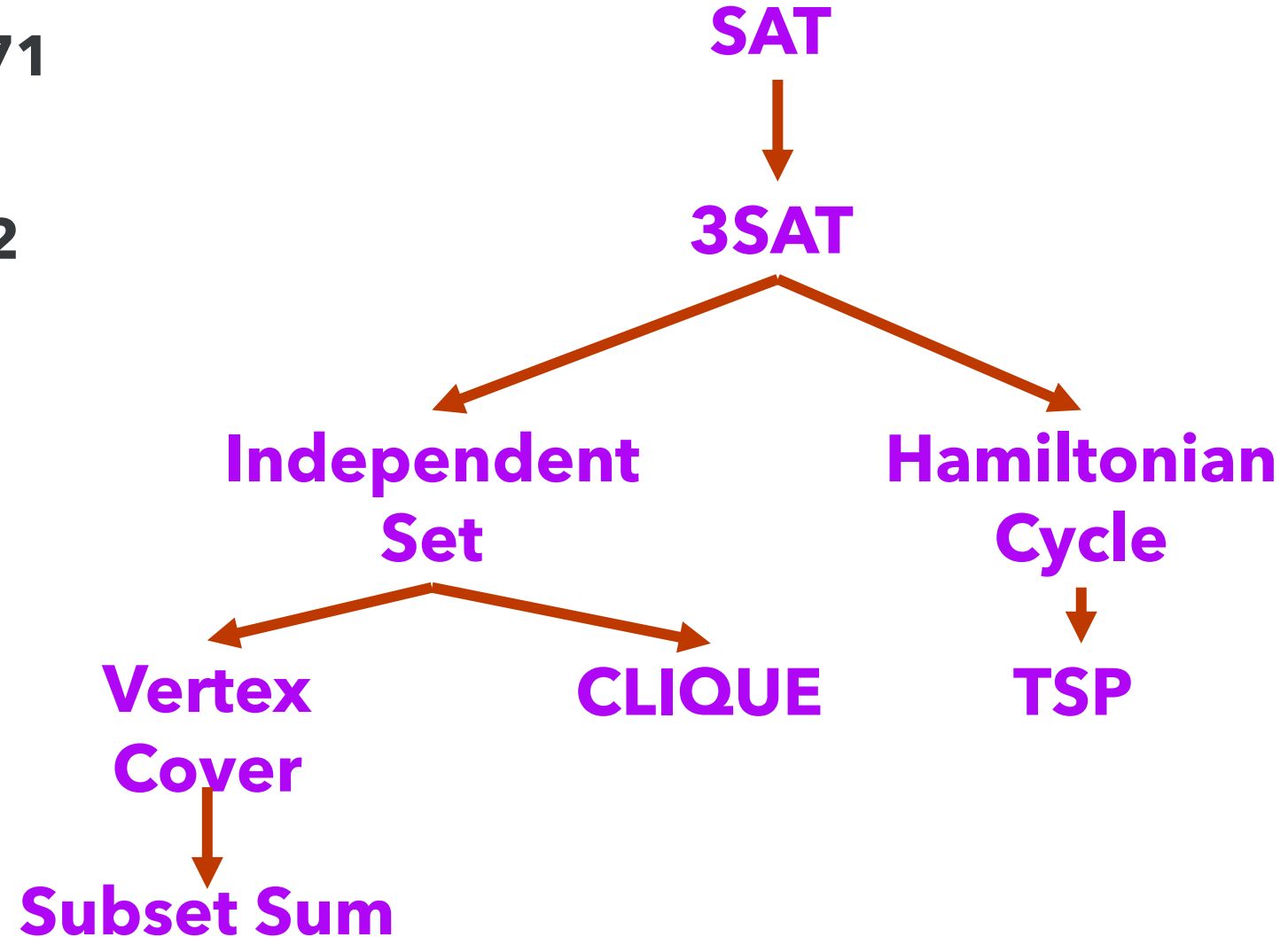
NP-COMPLETENESS OF 3-SAT

- If C_r satisfiable, then C'_r is satisfiable.
- If either $a_1 = 1$ or $a_2 = 1$, then we can assign $y = 0$ and $C'_r = 1$ because: $C'_r = (a_1 \vee a_2 \vee 0) \wedge (1 \vee A)$ and one or both a_1, a_2 are 1.
- If $a_1 = 0, a_2 = 0$, then $A = 1$. By assigning $y = 1$, we can make C'_r satisfiable.
- Using the above procedure, $C_r \forall r$ where the number of literals is greater than 3 can be converted into clauses with at most 3 variables that are equisatisfiable. From the above proof, we can see that this takes polynomial time in the number of literals in every clause.
- Therefore, we can reduce the SAT to 3-SAT in polynomial time. From Cook's theorem, the **SAT is NP-Complete**. Hence **3-SAT is also NP-Complete**.

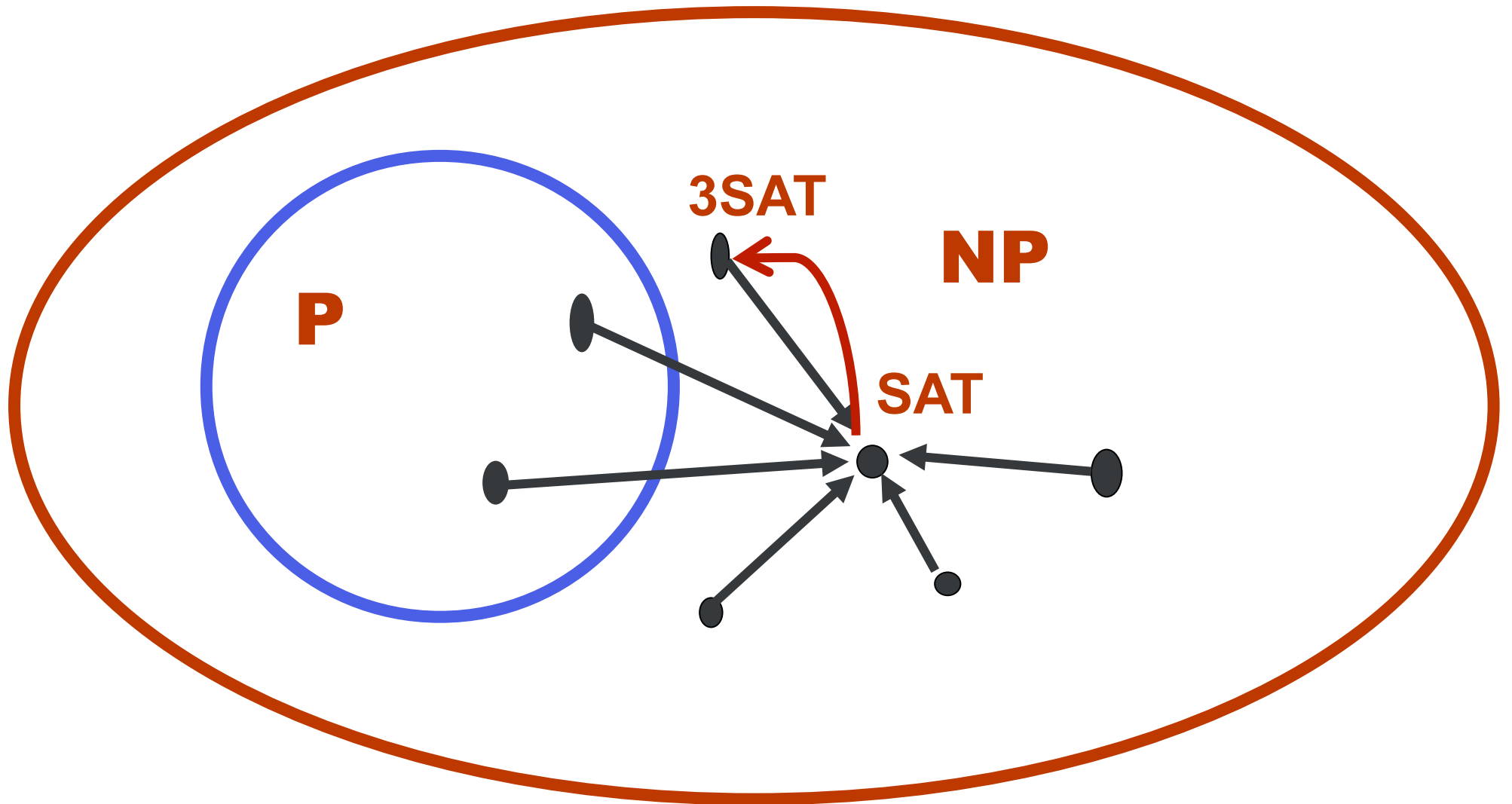
NP-COMPLETENESS PROBLEM

➤ Stephen Cook 1971

➤ Richard Karp 1972

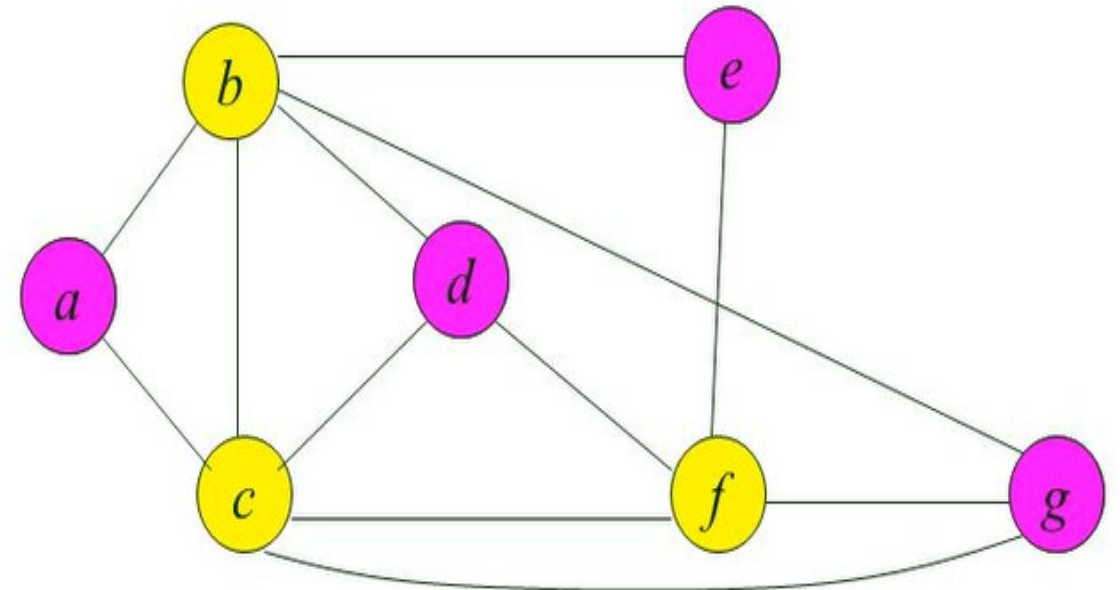


What we want to prove?



MAXIMUM INDEPENDENT SET

- Input: undirected Graph $G = (V, E)$,
- $A \subseteq V(G)$ is an independent set if no two vertices in A share an edge
- $\forall u, v \in A, (u, v) \notin E(G)$
- Output: size of the largest subset of vertices with no edges between them
- $A = \{a, d, e, g\}$ is an independent set
- $B = \{c, a, f\}$ is not



MAXIMUM INDEPENDENT SET

- Input: Graph $G = (V, E)$, integer $K \geq 1$, Does G have an independent set of size $\geq K$?
- Theorem IND-SET is NP Complete
- Proof:
 - IND-SET $\in$ NP. Below is a non deterministic algorithm:
 - Pick K vertices from $V(G)$ to form a subset $A \subseteq V(G)$
 - Picking operation is non-deterministic $O(K)$ time
 - Check if $\forall u, v \in A, (u, v) \notin E(G)$
 - If so answer is YES, otherwise , answer NO – deterministic $O(K^2)$ time.
 - We show that 3SAT is polynomial time reducible to IND set. By constructing (G_ϕ, K) such that $\phi \in 3SAT \Leftrightarrow (G_\phi, K) \in IND - SET$

3SAT TO INDEPENDENT SET

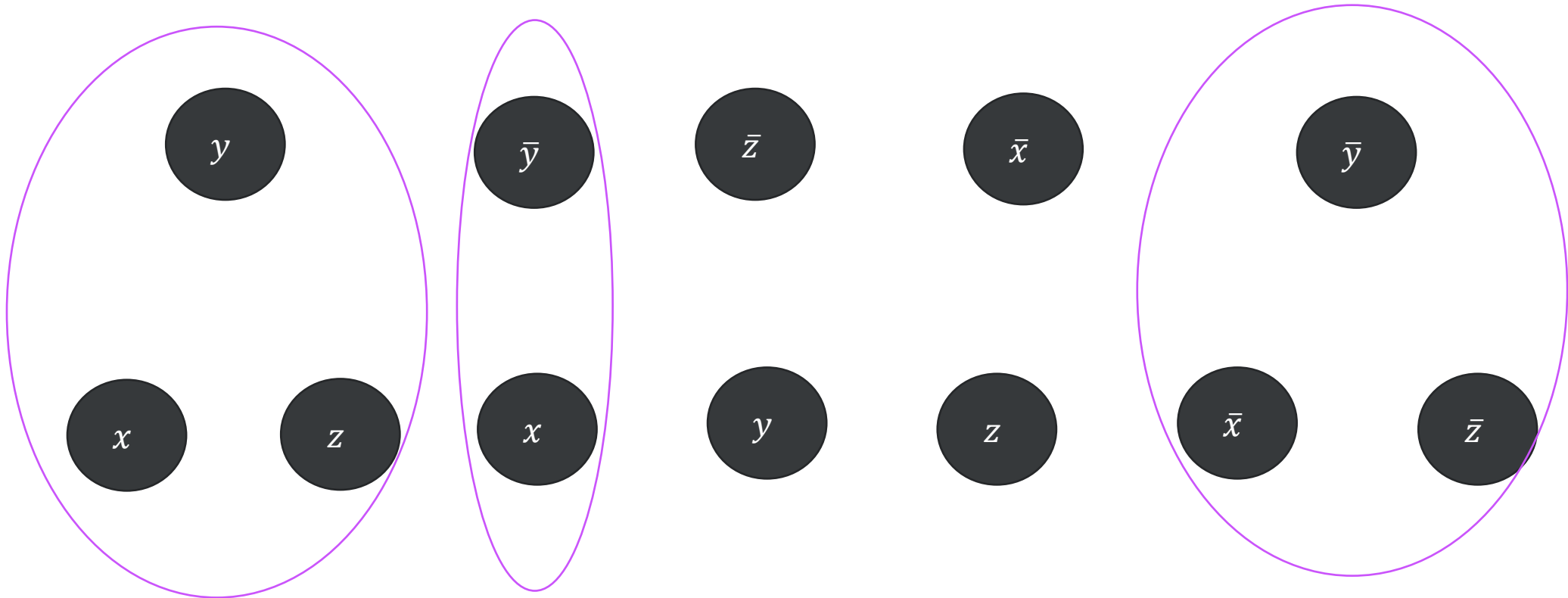
- 3 SAT
- Input: Formula in 3 CNF (a collection of clauses each having at most three literals)
- Output: An assignment of Boolean values to the variables of F satisfying all clauses, if exists.
- To reduce 3 SAT to independent Set
 - Design a polynomial time algorithm that, given a 3 CNF formula F , output a graph G and an integer b , such that:
 - F is satisfiable, iff G has an independent set of size at least b .

3SAT TO INDEPENDENT SET

- Reduction means
- We need to find an assignment of Boolean values of variables, such that each clause contains at least one satisfied literal.
- Example:
 - Setting $x = 1, y = 1, z = 1$ satisfies a formula $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})$
 - Setting $x = 1, y = 0, z = 0$ satisfies a formula $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})$
- Alternatively, we need to select at least one literal from each clause, such that the set of selected literals is consistent: it does not contain a literal l together with its $\bar{l}$.
- Example: $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})$
- Consistent: $\{x, x, \bar{z}\}, \{x, x, y\}, \{x, \bar{y}, \bar{z}\}$ Inconsistent: $\{y, \bar{y}, \bar{z}\}, \{z, x, \bar{z}\}$

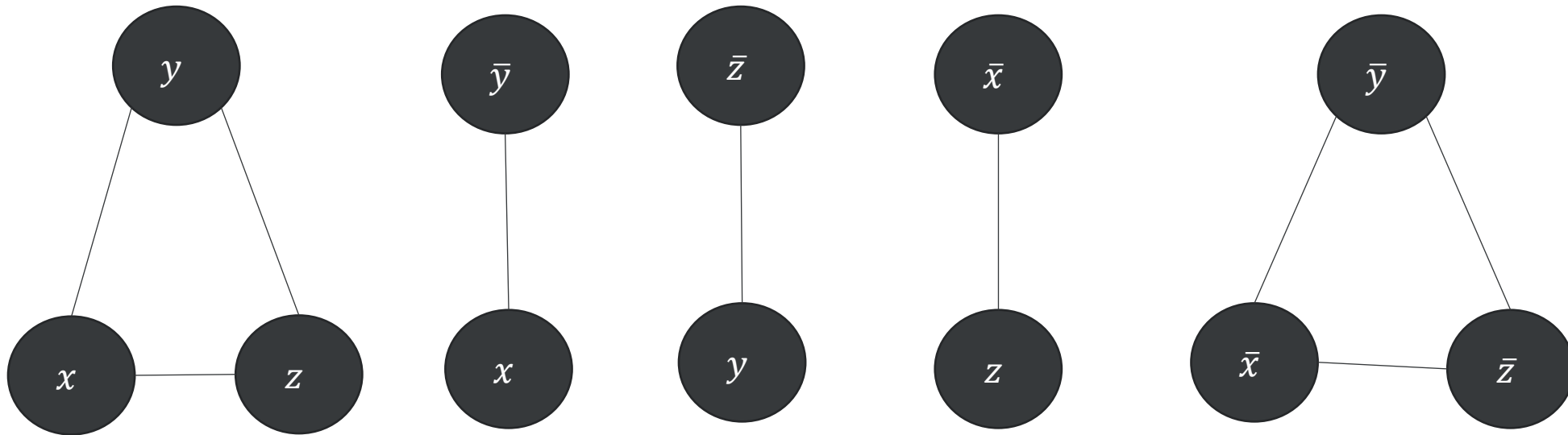
3SAT TO INDEPENDENT SET

- Using Alternative statement
- Example: $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})(z \vee \bar{x})(\bar{x} \vee \bar{y} \vee \bar{z})$



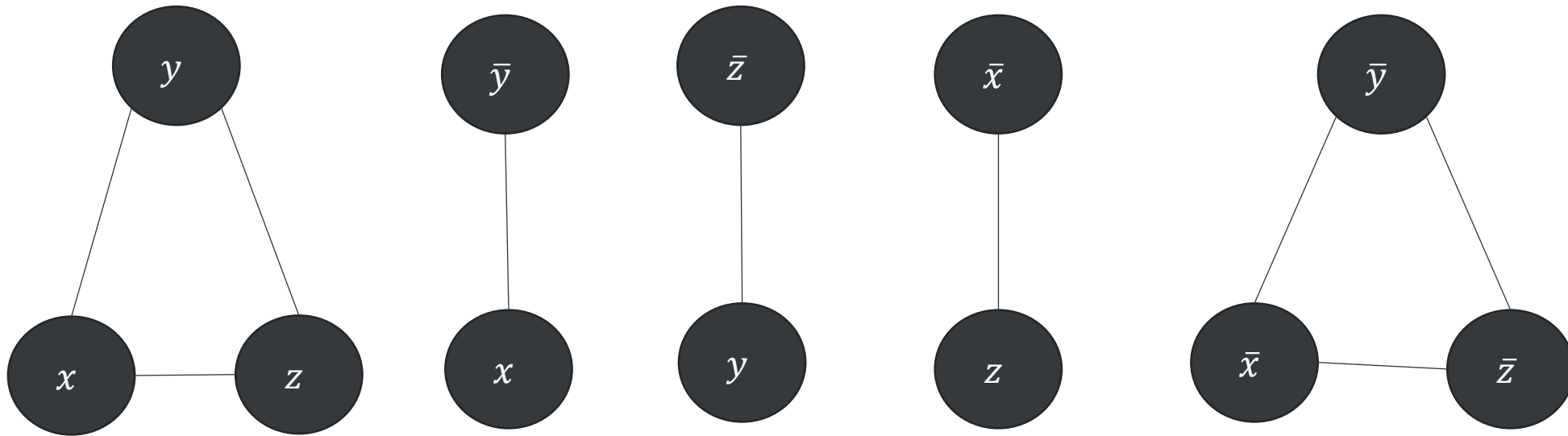
3SAT TO INDEPENDENT SET

- Example: $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})(z \vee \bar{x})(\bar{x} \vee \bar{y} \vee \bar{z})$
- Join the vertices inside the clause



3SAT TO INDEPENDENT SET

- Example: $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})(z \vee \bar{x})(\bar{x} \vee \bar{y} \vee \bar{z})$
- Find the large independent set for these clause, set must has at most vertex from each block,

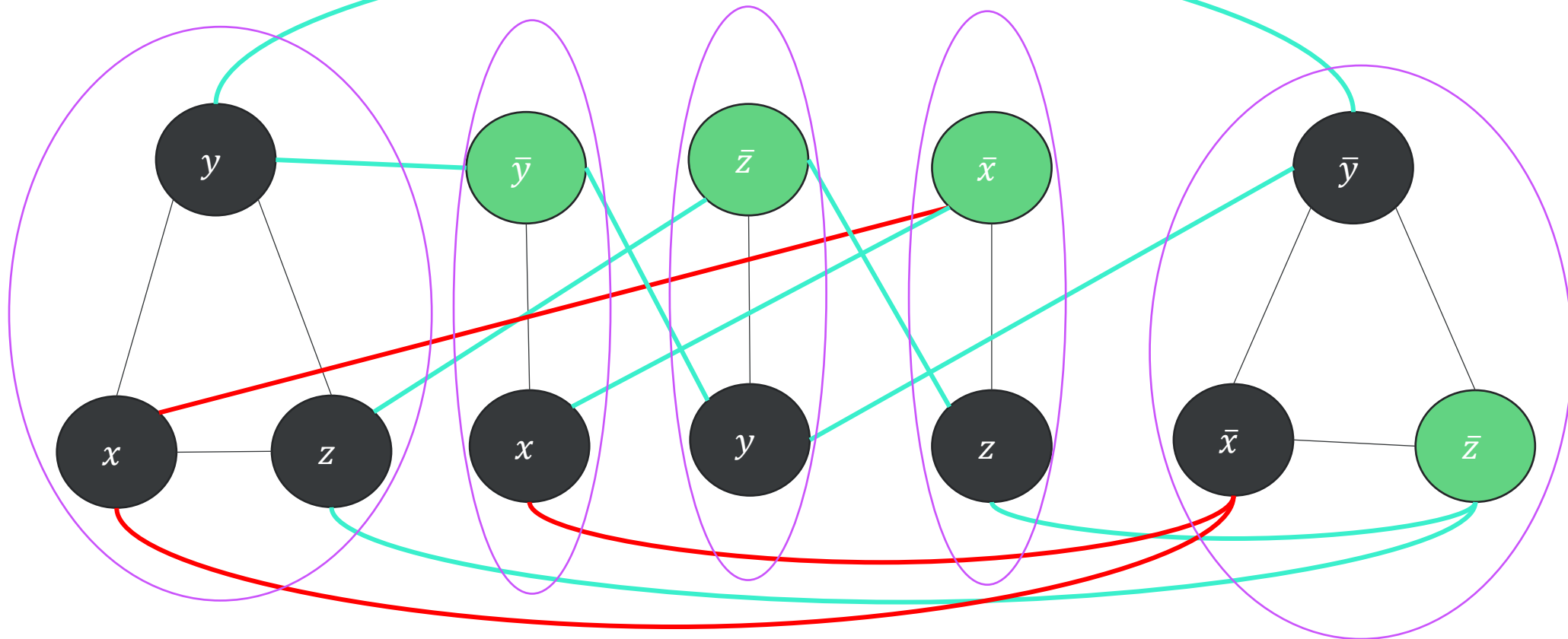


3SAT TO INDEPENDENT SET

- Example: $(x \vee y \vee z)(x \vee \bar{y})(y \vee \bar{z})(z \vee \bar{x})(\bar{x} \vee \bar{y} \vee \bar{z})$
- So selecting a vertex from each of these blocks into an independent set corresponds to selecting a literal from the corresponding clause, which is going to be satisfied in this clause.
- In each clause we need to select a literal which is going to satisfy this clause. So we select the corresponding vertex from a block. However, recall that we need to select a consistent set of literals.
- i.e we do not want to select x from some clause and not x from some other clause.
- To find our set of selected vertices to be labeled with a consistent set of literals, we just add all possible edges between all pairs of complementary literals.

3SAT TO INDEPENDENT SET

- Namely, whenever we have a vertex labeled with x , we join x with $\bar{x}$ by an edge in another clause. Like this we connect all literals with its complementary of another clause.



3SAT TO INDEPENDENT SET

- Now, the graph contains an independent set of sides, since the formula is satisfied iff the resulting graph has independent set of size 5.
- So we can select one literals from each clause such that the resulting set of literals is consistent. But this also means that from each of these five block, we can select exactly the same vertex that corresponds to the selected literals in this formula.
- If the formula is satisfiable, then in each block we can select on vertex and we know that they are not going to be joined by edges, because vertices from different blocks are only joined with their complementarians. Vice versa if an independent set of size 5 formed from graph then the given formula is satisfiable.

3SAT TO INDEPENDENT SET

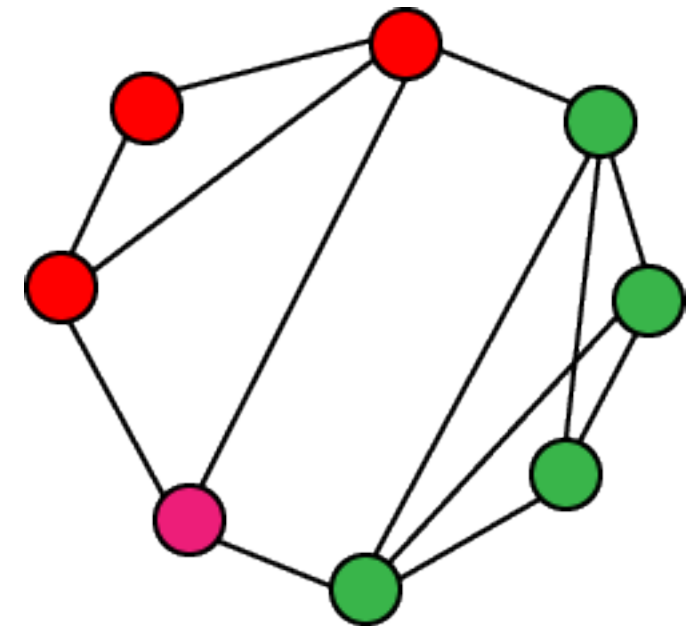
- Transforming an instance
- **For each clause of the input formula F , introduce three (or two or one) vertices in G labeled with the literals of this clause. Join every two of them.**
- So constructing a graph G , and this forces every independent set in this graph to contain at most one vertex from each block to responding to a single clause.
- For each independent set need to labeled with a set of literals which is consistent.
- **Join every pair of vertices labeled with complementary literals.**
- **F is satisfiable iff G has independent set of size equal to the number of clause in F , it takes polynomial time.**

3SAT TO INDEPENDENT SET

- Transforming a solution
- Given a solution S for G , just read the labels of the vertices from S to get a satisfying assignment of F (take polynomial time)
- If there is no solution for G , then F is unsatisfiable: indeed, a satisfying assignment for F would give a required independent set in G .

CLIQUE

- **Definition:** - In Clique, every vertex is directly connected to another vertex, and the number of vertices in the Clique represents the Size of Clique.
- **CLIQUE COVER:** - Given a graph G and an integer k , can we find k subsets of vertices $V_1, V_2 \dots V_k$, such that $\bigcup V_i = V$, and that each V_i is a clique of G .



Clique cover (size=3)

3CNF $\leq_p$ CLIQUE

- Proof:-For the successful conversion from 3CNF to Clique, you have to follow the two steps:-
- Draw the clause in the form of vertices, and each vertex represents the literals of the clauses.
 - They do not complement each other
 - They don't belong to the same clause
 - In the conversion, the size of the Clique and size of 3CNF must be the same, and you successfully converted 3CNF into Clique within the polynomial time

CLIQUE $\leq_p$ 3CNF

- **Proof:** - As you know that a function of K clause, there must exist a Clique of size k. It means that P variables which are from the different clauses can assign the same value (say it is 1). By using these values of all the variables of the CLIQUES, you can make the value of each clause in the function is equal to 1
- **Example:** - You have a Boolean function in 3CNF:-
- $(X+Y+Z)(X+Y+Z')(X+Y'+Z)$
- After Reduction/Conversion from 3CNF to CLIQUE, you will get P variables such as: $x+y=1$, $x+z=1$ and $x=1$
- Put the value of P variables in equation (i)
- $(1+1+0)(1+0+0)(1+0+1) = (1)(1)(1) = 1$ output verified

CLIQUE $\in$ NP

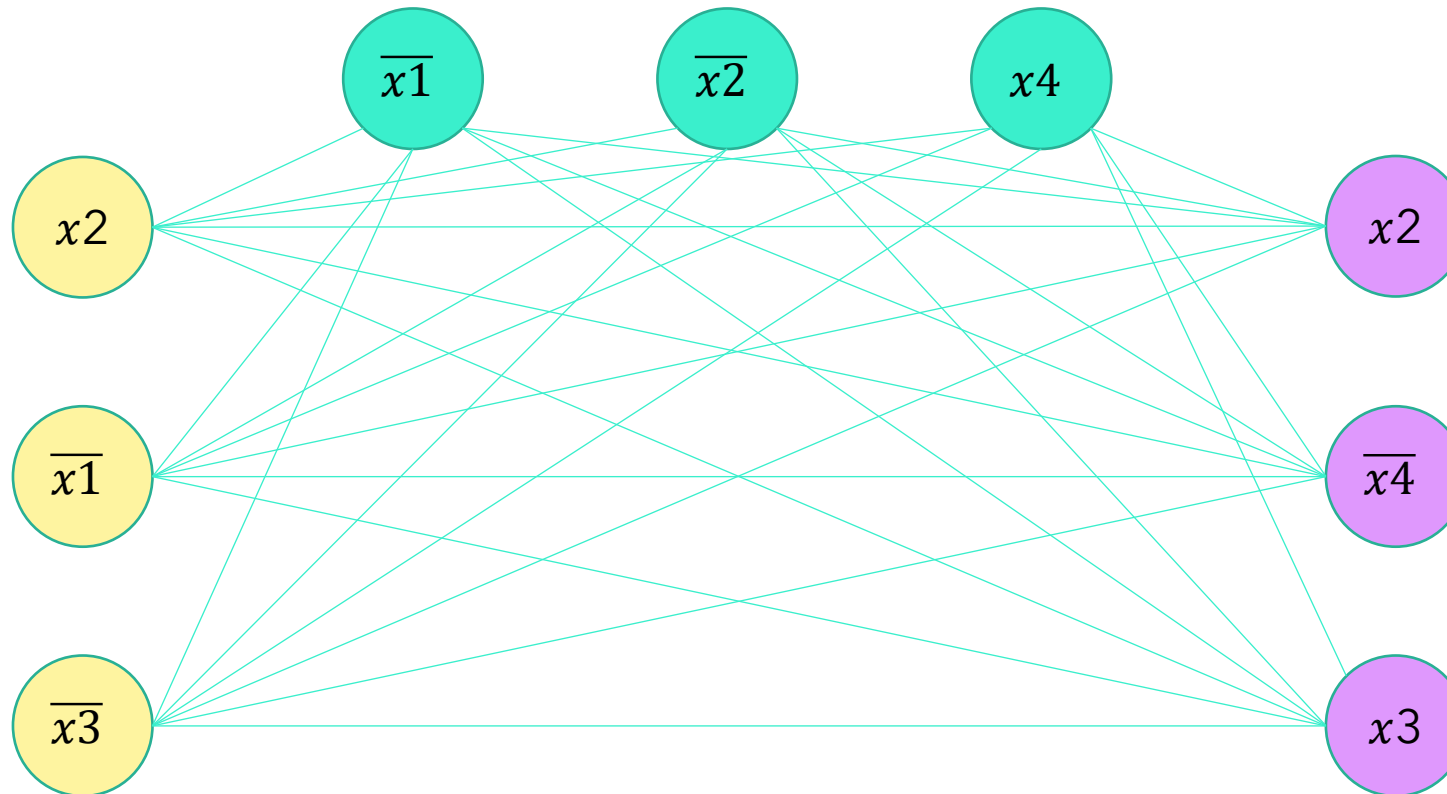
- Proof: - As you know very well, you can get the Clique through 3CNF and to convert the decision-based NP problem into 3CNF you have to first convert into SAT and SAT comes from NP.
- So, concluded that CLIQUE belongs to NP.
- Proof of NPC:-
 - Reduction achieved within the polynomial time from 3CNF to Clique
 - And verified the output after Reduction from Clique To 3CNF above
 - So, concluded that, if both Reduction and verification can be done within the polynomial time that means Clique also in NPC.

3SAT TO CLIQUE

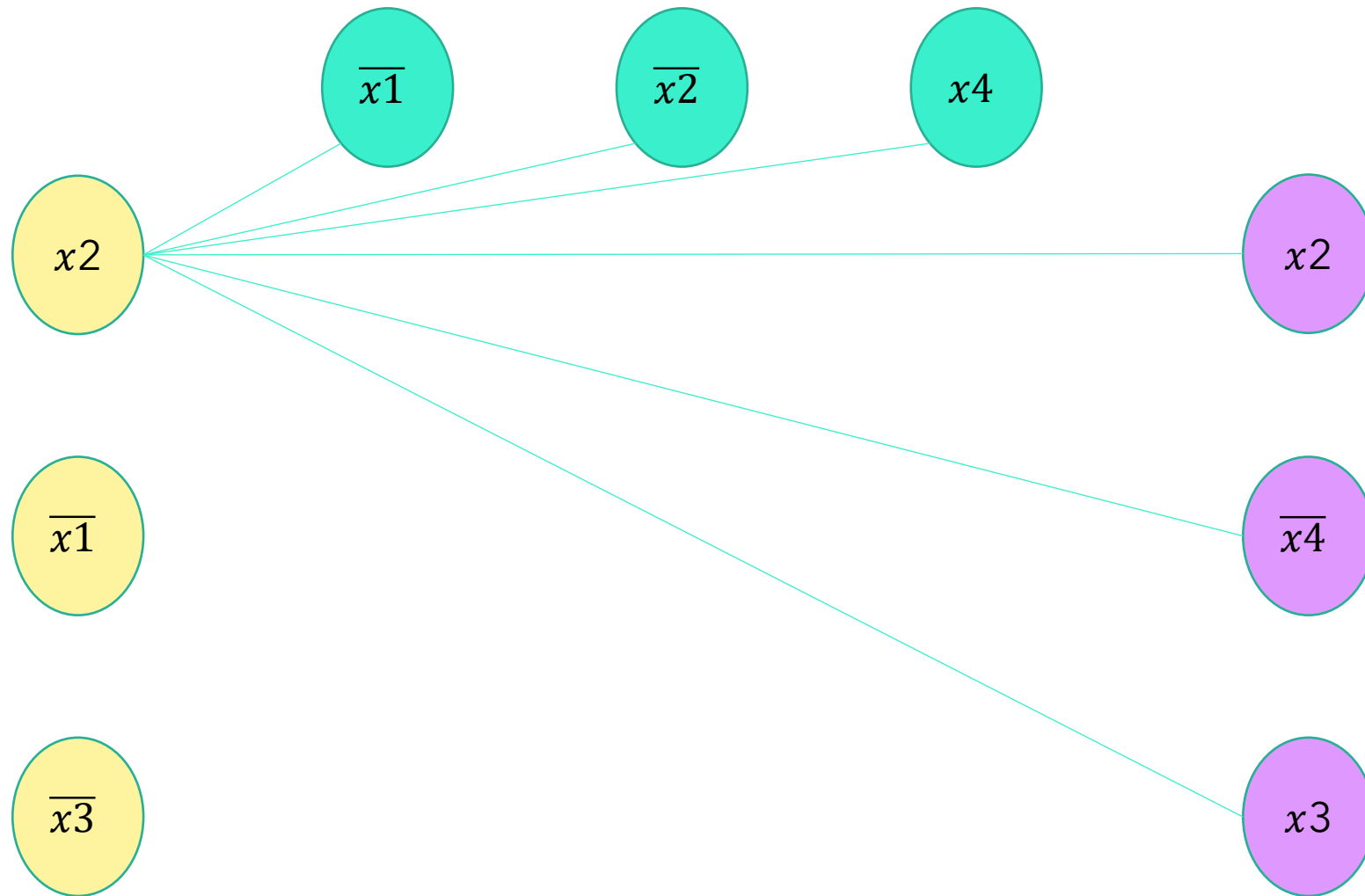
- A formula F of k three literal clause can be reduced to a k -clique problem in following ways
- Construct a graph G of k clusters with a maximum of 3 nodes in each cluster.
- Each cluster corresponds to a clause in F .
- Each node in a cluster is labeled with a literal from the clause.
- An edge is put between all pairs of nodes in different clusters except for pairs of the form x , not x .
- No edge is put between any pair of nodes in the same cluster.

3SAT TO CLIQUE

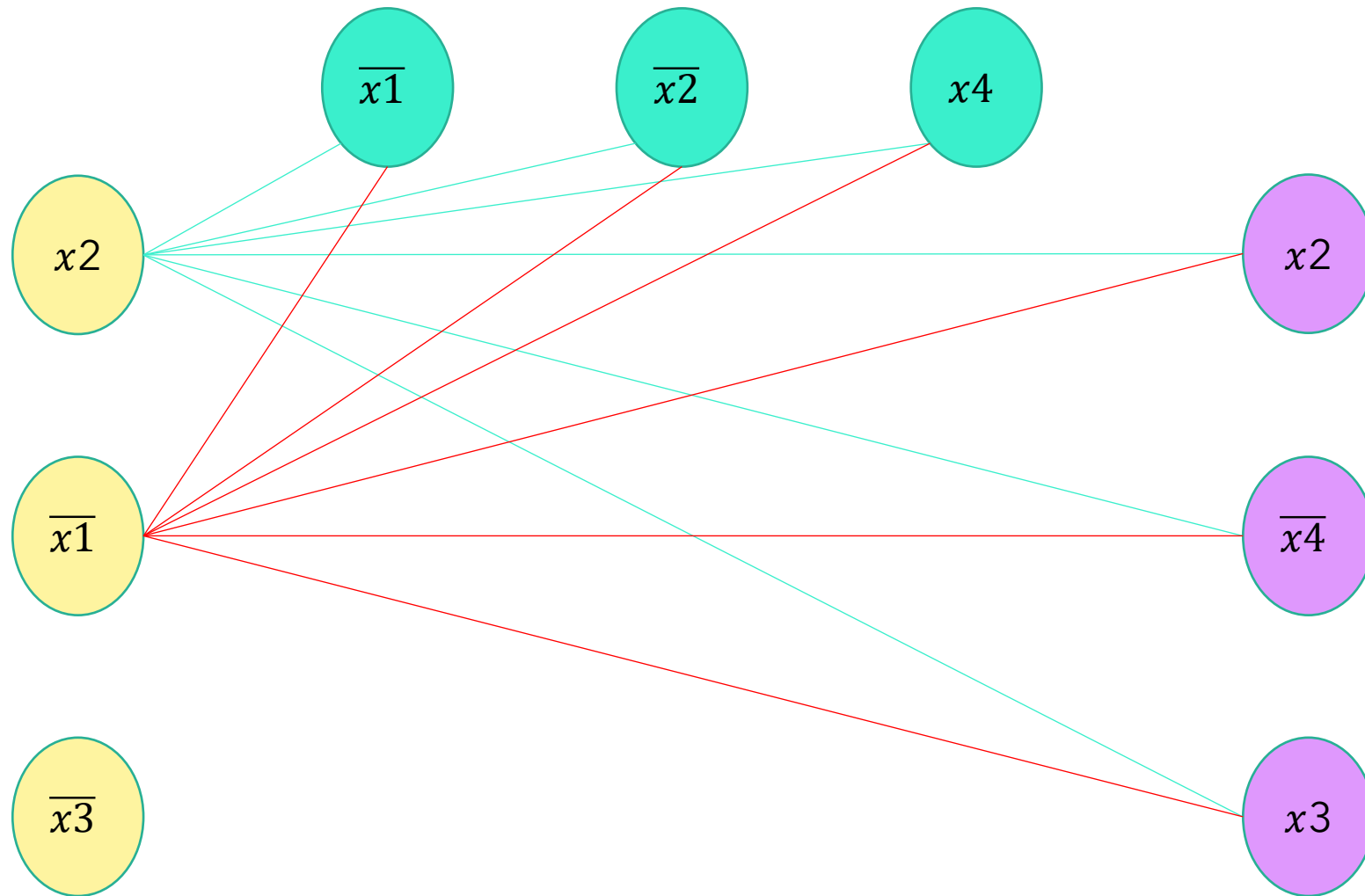
- Constructing of cluster of nodes corresponding to clause in 3CNF
- $F = (x_2 + x_1 + \overline{x_3})(\overline{x_1} + \overline{x_2} + x_4)(x_2 + \overline{x_4} + x_3)$



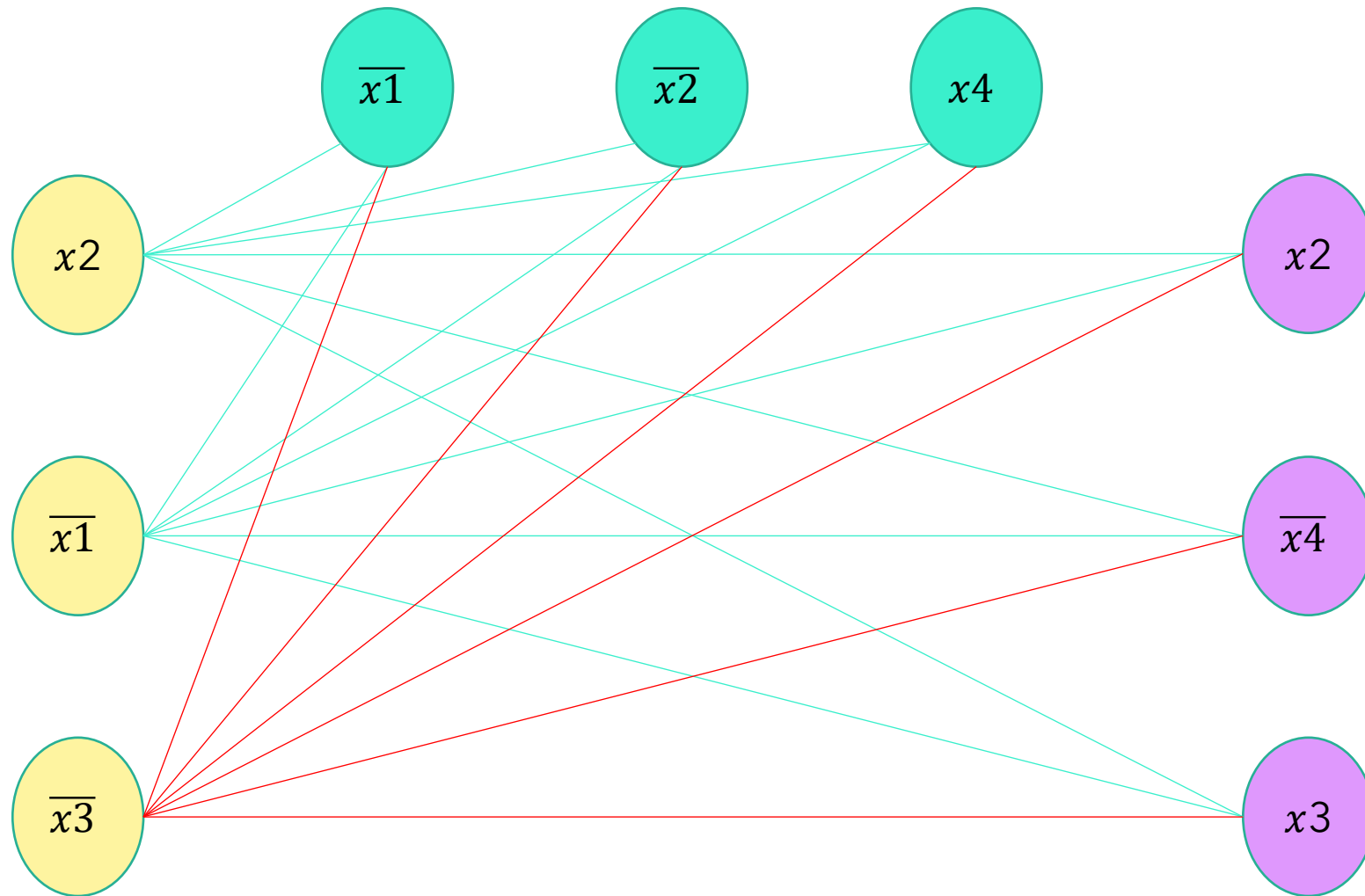
3SAT TO CLIQUE



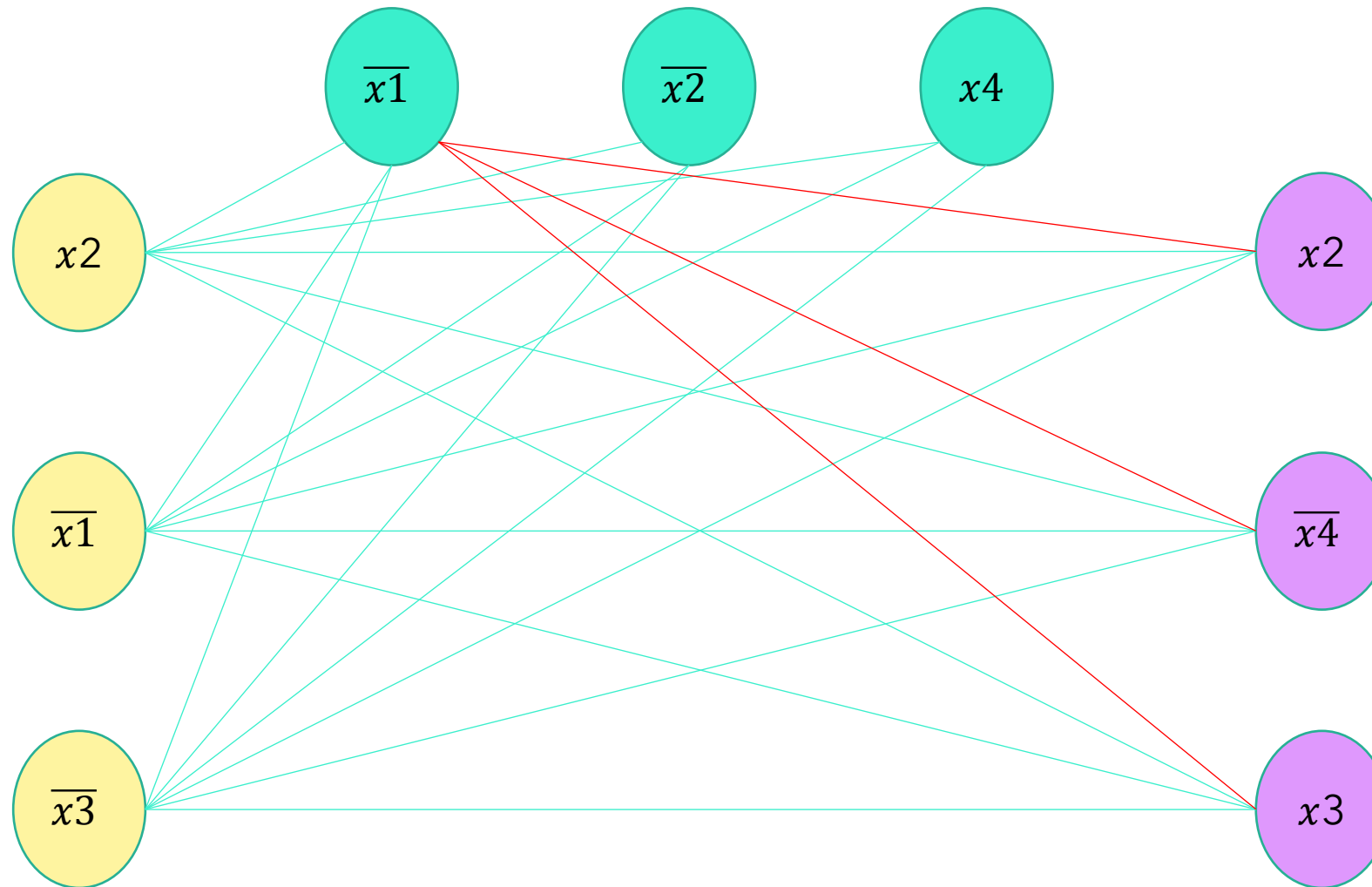
3SAT TO CLIQUE



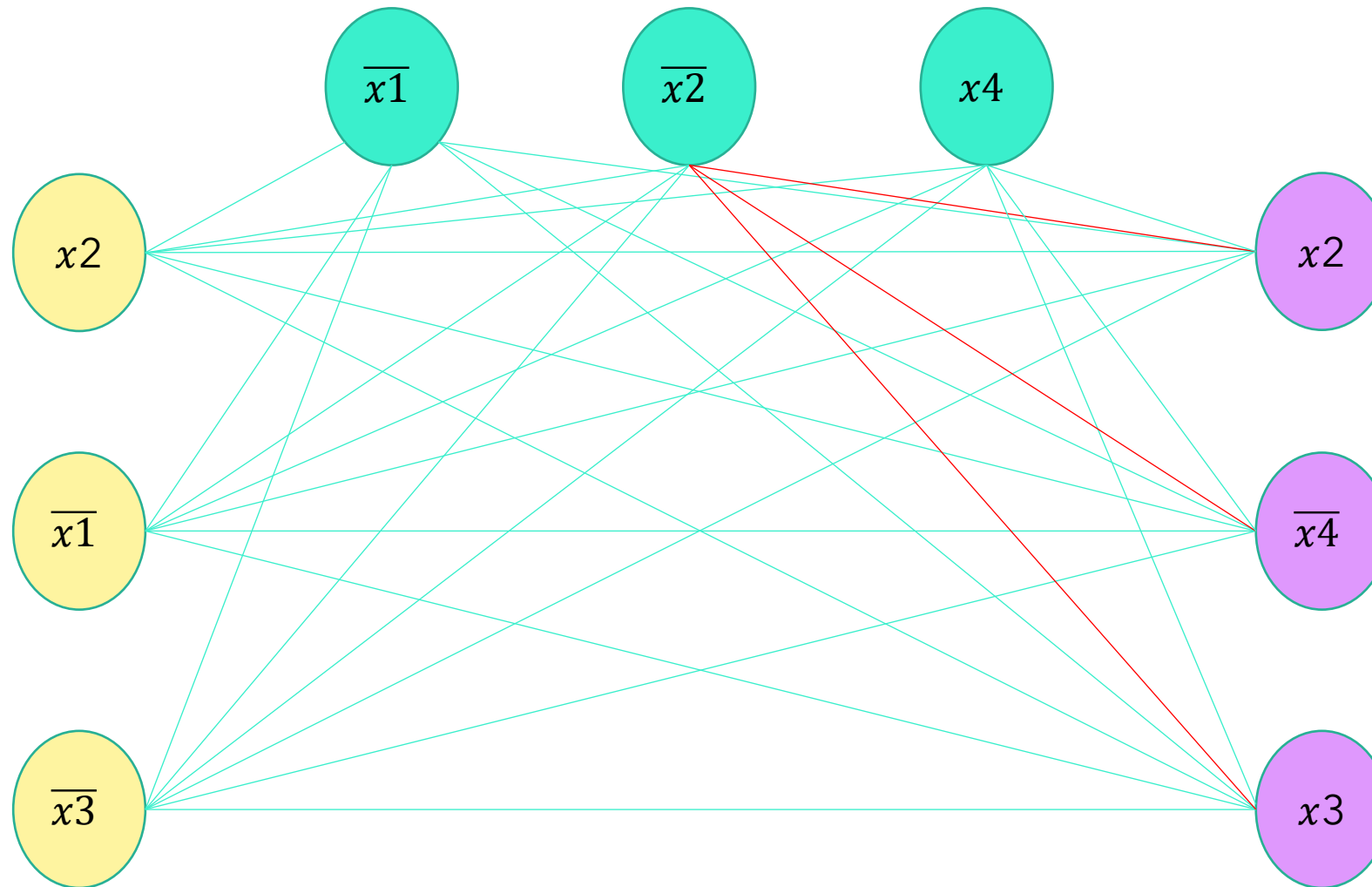
3SAT TO CLIQUE



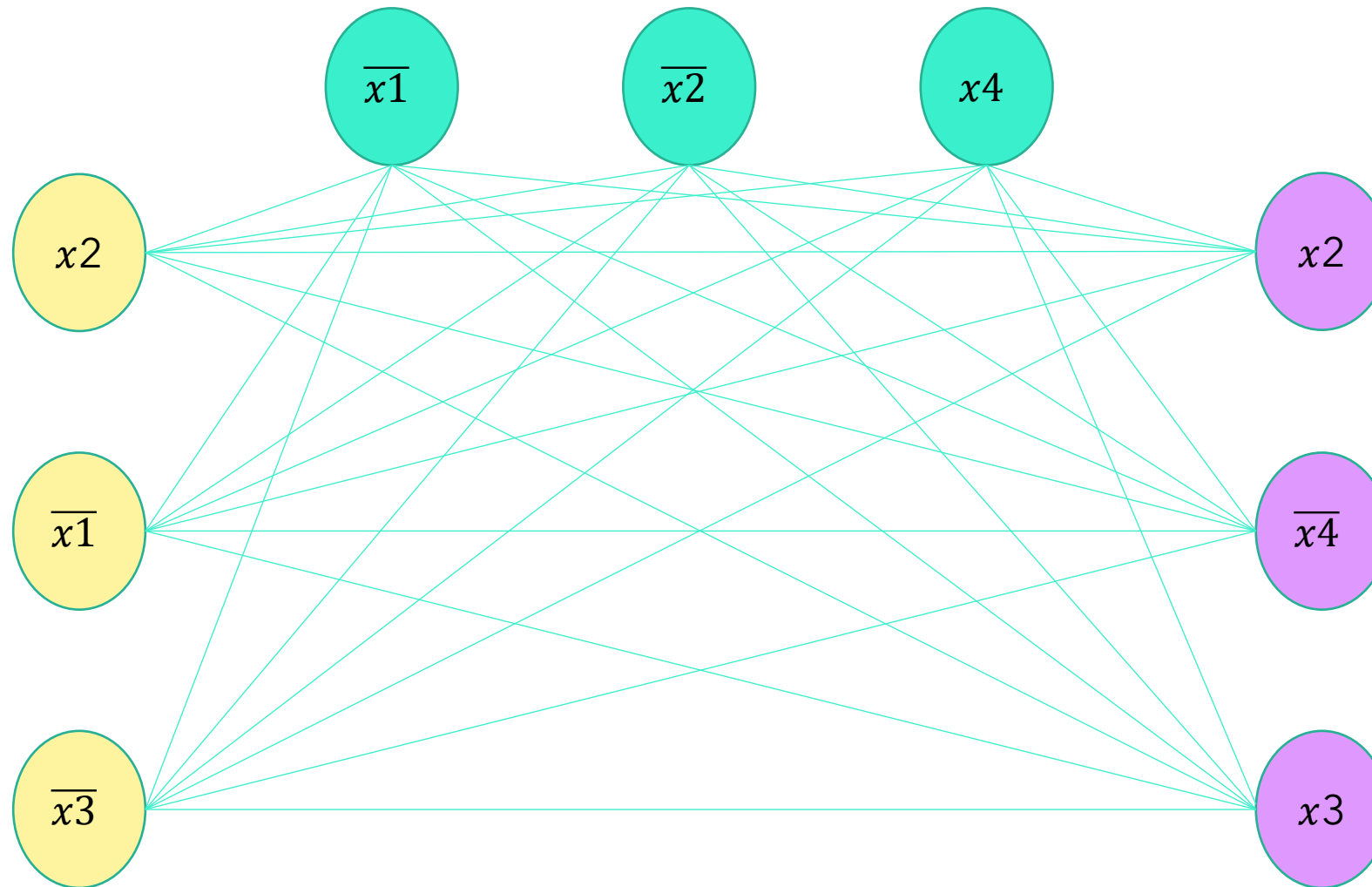
3SAT TO CLIQUE



3SAT TO CLIQUE

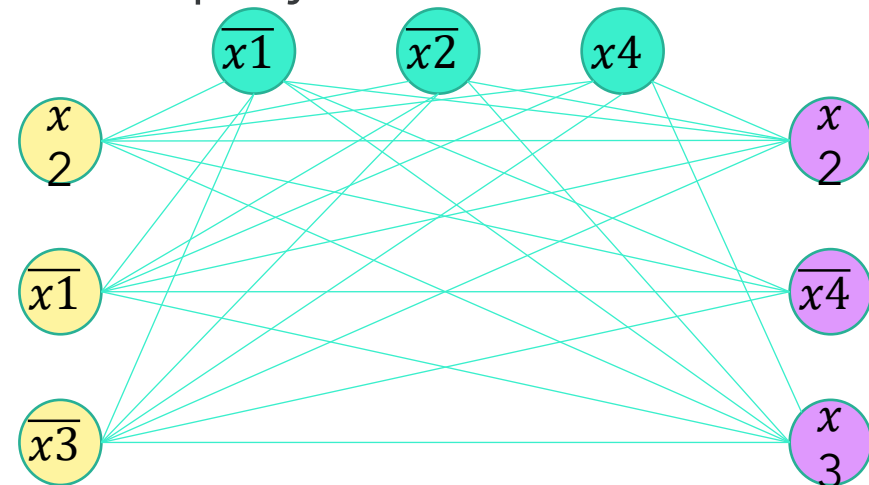


3SAT TO CLIQUE



3SAT TO CLIQUE

- About graph:
- If two nodes in the graph are connected, the corresponding literals can be simultaneously be assigned True.
- If two literal, not from the same clause can be assigned True simultaneously, the nodes corresponding to these literals in the graph are connected.
- Construction of the graph can be performed in polynomial time.

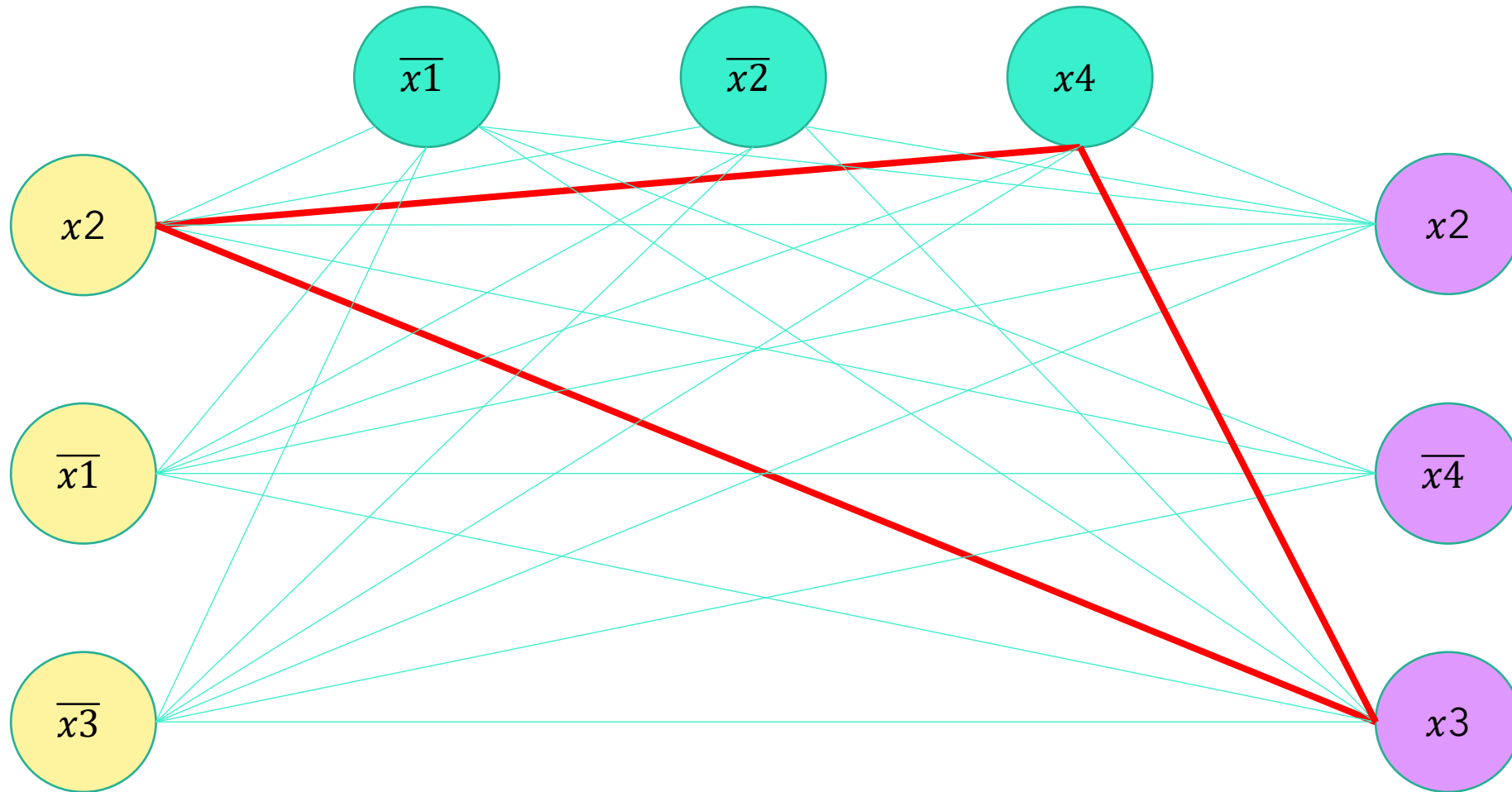


3SAT TO CLIQUE

- Reduction:
- G has k -clique iff F is satisfiable.
- **If G has k -clique**, the clique has exactly one node from each cluster. Because no two nodes from the same cluster are connected to each other, hence they can never be a part of the same clique.
- All nodes in a clique are connected, hence all corresponding literals can be assigned True simultaneously. Each literals belong to one of k -clause. Hence **F is satisfiable**.
- **If F is satisfiable**, let A be satisfying assignment. Select from each clause a literal that is True in A to construct a set S . $|S|=k$. since no two literals in A are from same clause and all of them are simultaneously True, all the corresponding nodes in the graph are connected to each other, forming a k -clique. Hence **the graph has a k -clique**.

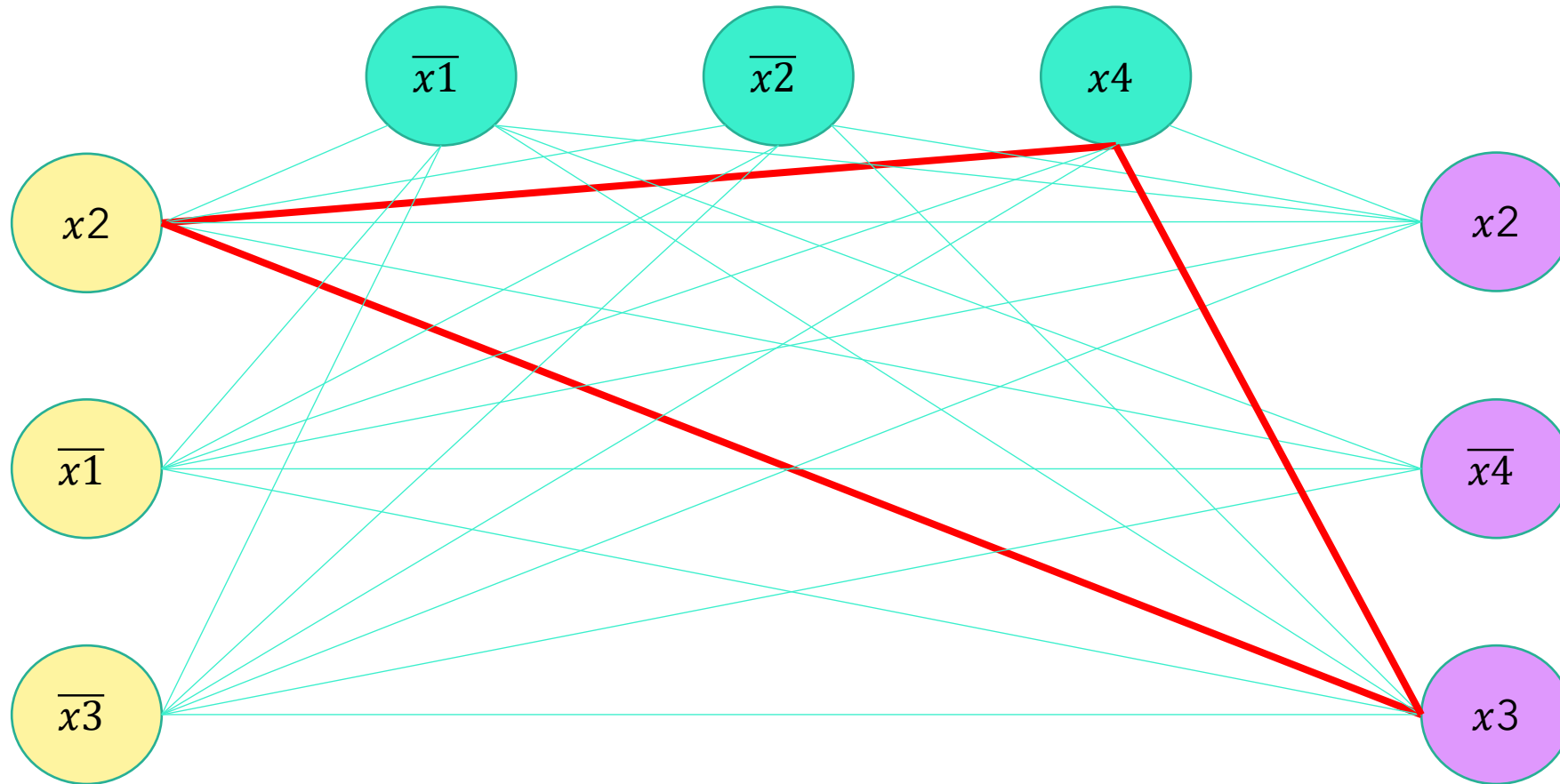
3SAT TO CLIQUE

- The corresponding assignment $x_2 = x_3 = x_4 = \text{True}$.



3SAT TO CLIQUE

- $F = (x_2 + x_1 + \overline{x_3})(\overline{x_1} + \overline{x_2} + x_4)(x_2 + \overline{x_4} + x_3)$, F is True for corresponding assignment $x_2=x_3=x_4=\text{True}$.





THANK YOU

Module 7

BCSE204L - Design and Analysis of Algorithm